

Graph Search

Comp Sci 214, Spring 2025

Don't forget to check in on Youhere!

First Midterm Recap

- *See companion document on Canvas*
 - ▶ Should answer most questions you have
 - ▶ Protocol for exam-related communications
- Your graded exam is on Crowdmark
- For modifiers, and how you're doing overall: use your hw1

First Midterm Recap

- *See companion document on Canvas*
 - ▶ Should answer most questions you have
 - ▶ Protocol for exam-related communications
- Your graded exam is on Crowdmark
- For modifiers, and how you're doing overall: use your hw1
- If you're pleased with your performance
 - ▶ Don't let your guard down; we're not done!
- If you're disappointed with your performance
 - ▶ Don't despair; we're not done!
 - ▶ But you'll likely want to change how you do things
- **Keep in mind:** ultimately, that exam is but a small thing
 - ▶ Only one exam, for *one* of your classes
 - ▶ Don't lose track of the big picture!

First Midterm Recap

- *See companion document on Canvas*
 - ▶ Should answer most questions you have
 - ▶ Protocol for exam-related communications
- Your graded exam is on Crowdmark
- For modifiers, and how you're doing overall: use your hw1

- If you're pleased with your performance
 - ▶ Don't let your guard down; we're not done!
- If you're disappointed with your performance
 - ▶ Don't despair; we're not done!
 - ▶ But you'll likely want to change how you do things
- **Keep in mind:** ultimately, that exam is but a small thing
 - ▶ Only one exam, for *one* of your classes
 - ▶ Don't lose track of the big picture!

- **Second midterm**
 - ▶ Thursday June 5th (last class meeting)
 - ▶ Not cumulative (but builds on some old concepts)
 - ▶ Graph ADTs/DSs and onwards

Heads up: Tuesday Deadlines

Second half of the class has deadlines on *Tuesdays*

Homework 3 due *tonight*

Back to Graphs

Things we can do with graphs

- Take an action at every node reachable from some node v
 - ▶ E.g., collect garbage from every house in a neighborhood

Things we can do with graphs

- Take an action at every node reachable from some node v
 - ▶ E.g., collect garbage from every house in a neighborhood
- Ask whether there is a path from v to u
 - ▶ E.g., with a social graph, can my great-grandmother find a chain of people who can help her meet the pope?

Things we can do with graphs

- Take an action at every node reachable from some node v
 - ▶ E.g., collect garbage from every house in a neighborhood
- Ask whether there is a path from v to u
 - ▶ E.g., with a social graph, can my great-grandmother find a chain of people who can help her meet the pope?
(True story!)



Things we can do with graphs

- Take an action at every node reachable from some node v
 - ▶ E.g., collect garbage from every house in a neighborhood
- Ask whether there is a path from v to u
 - ▶ E.g., with a social graph, can my great-grandmother find a chain of people who can help her meet the pope?
(True story!)
- Ask whether there are any cycles in a graph
 - ▶ E.g., given an electric circuit diagram, is the circuit closed?
- We can accomplish these three tasks (and more!) with a *graph search* algorithm

Things we can do with graphs

- Take an action at every node reachable from some node v
 - ▶ E.g., collect garbage from every house in a neighborhood
- Ask whether there is a path from v to u
 - ▶ E.g., with a social graph, can my great-grandmother find a chain of people who can help her meet the pope?
(True story!)
- Ask whether there are any cycles in a graph
 - ▶ E.g., given an electric circuit diagram, is the circuit closed?
- We can accomplish these three tasks (and more!) with a *graph search* algorithm
- But first, we'll look at a restricted version of the problem
 - ▶ Let's start with one very restricted kind of graph: *trees*
 - ▶ And see simpler versions of graph search algos: *tree walks*

Tree walks

Tree walks (AKA tree traversals)

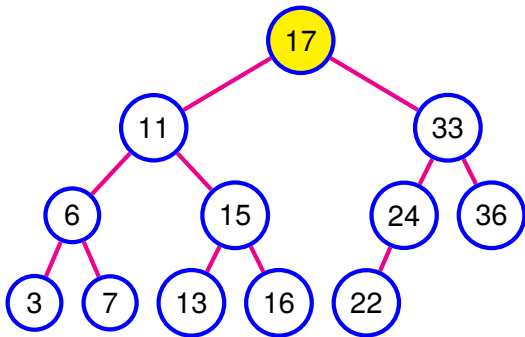
The goal of a tree walk is to *visit* every node in a tree.

- Necessary to solve a number of problems
 - ▶ Summing up the size of all the files in a filesystem directory
 - ▶ Executing a program with an interpreter (Cf. Comp Sci 321)
 - ▶ Etc.
- “Visiting” = taking an action at that node
 - ▶ Depends on what our algorithm is doing
 - ▶ E.g., adding the size of a file to the total
 - ▶ Here, we’ll leave it generic

We’ll start simple: tree walks on binary trees

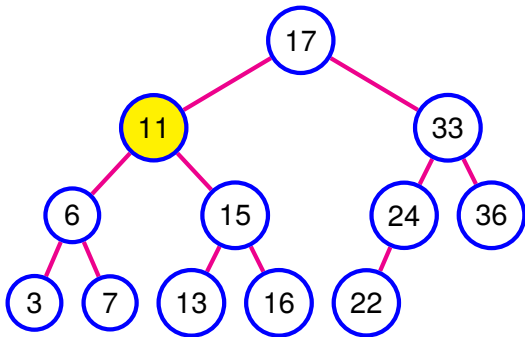
Tree walk example

- Start at the root of the tree



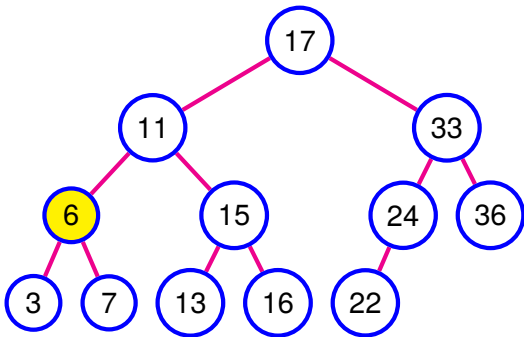
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.



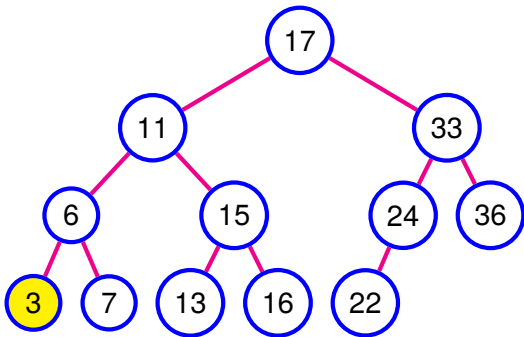
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.



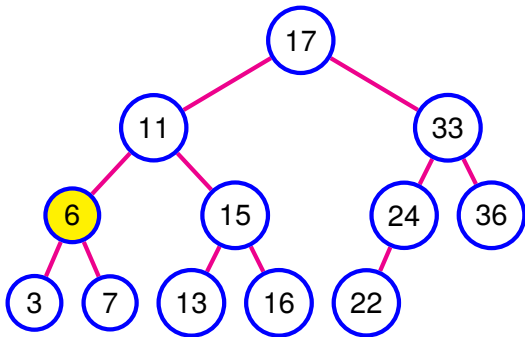
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up



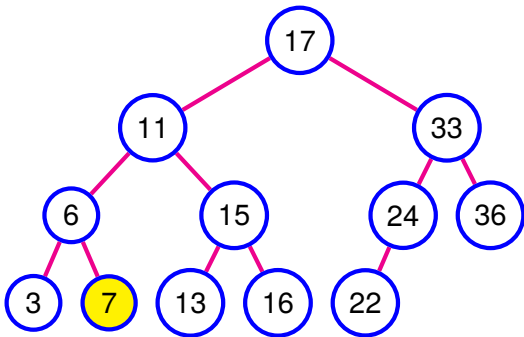
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up



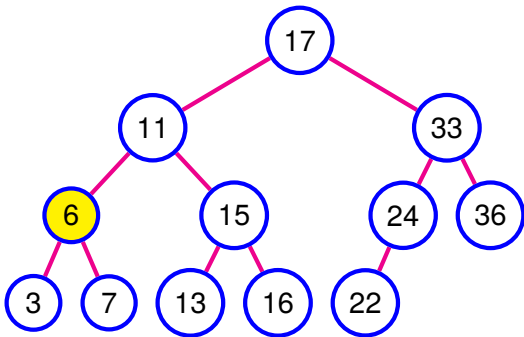
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)



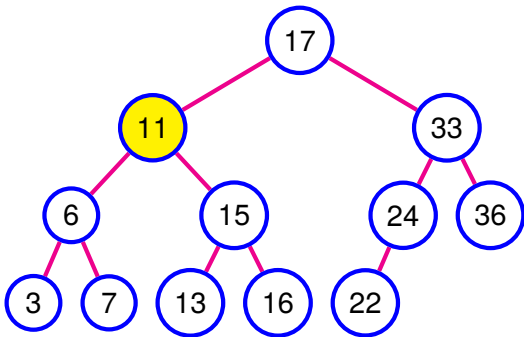
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



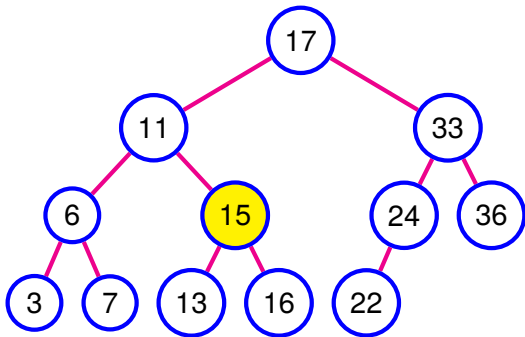
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



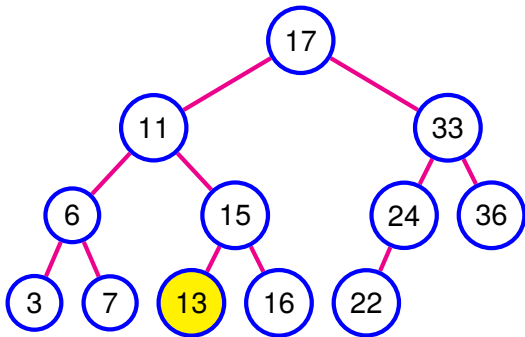
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



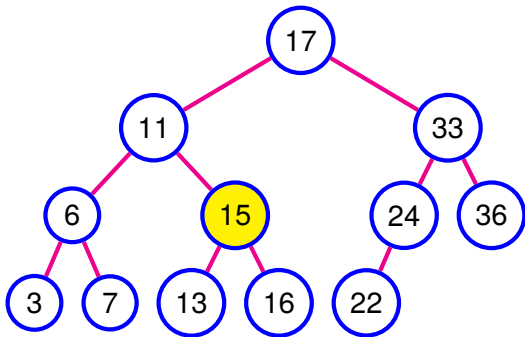
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



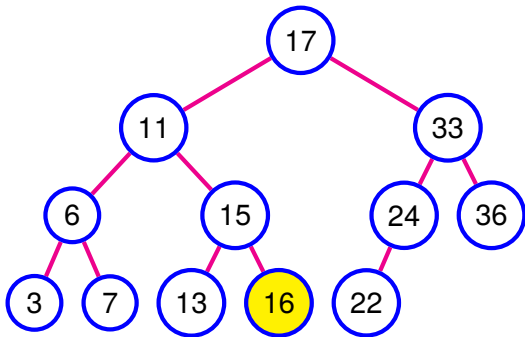
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



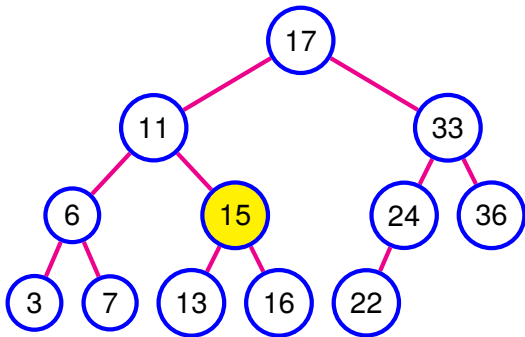
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



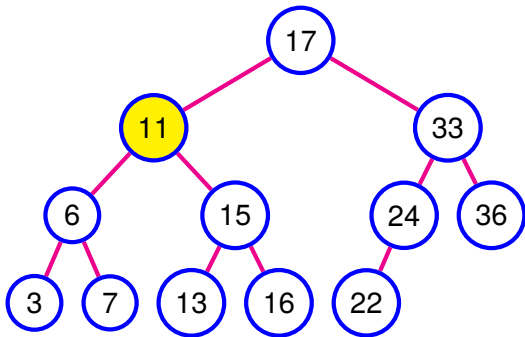
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



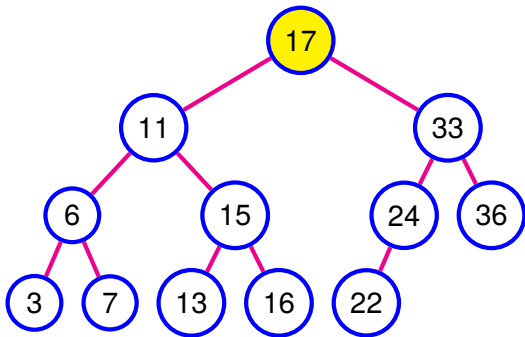
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



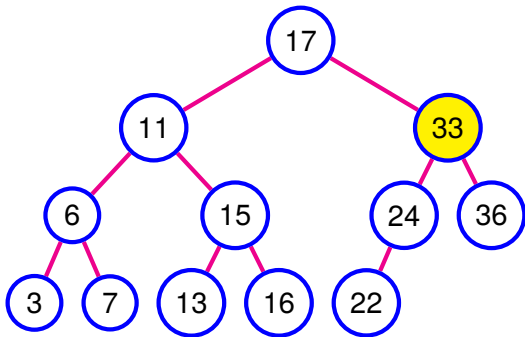
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



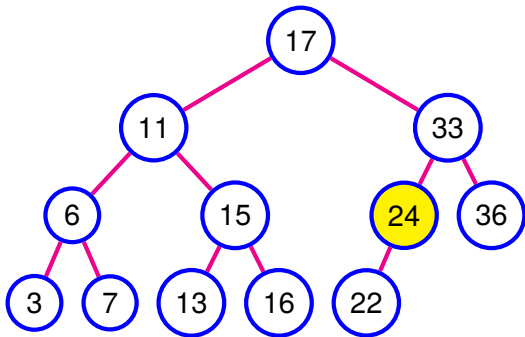
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



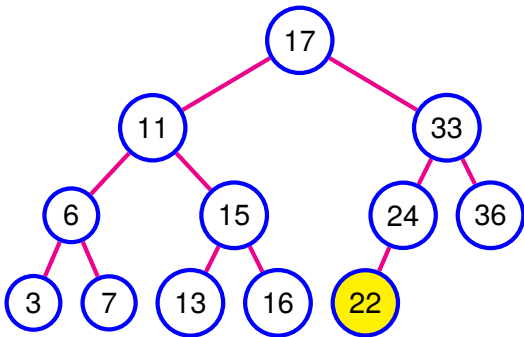
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



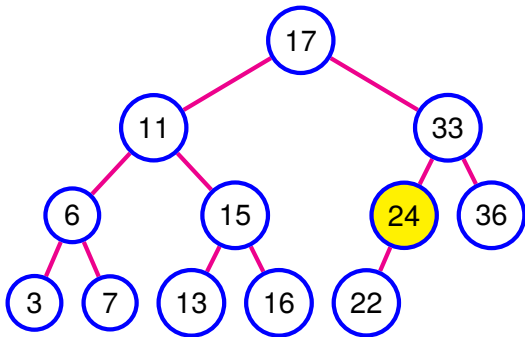
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



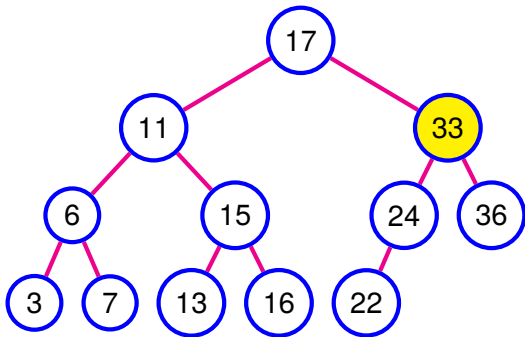
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



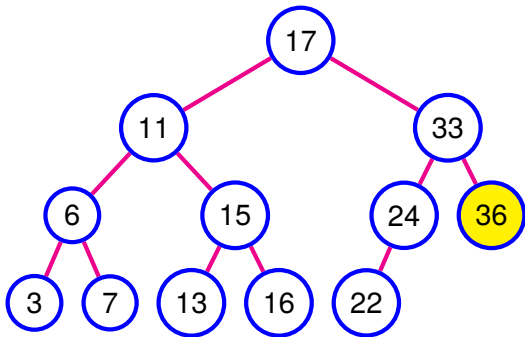
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



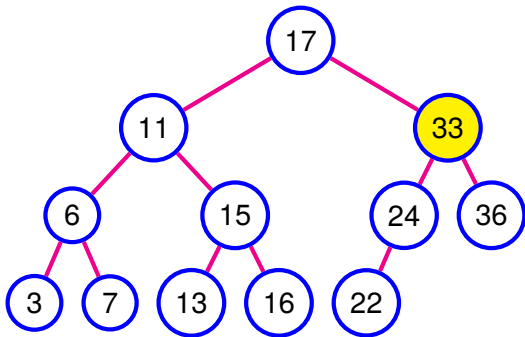
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



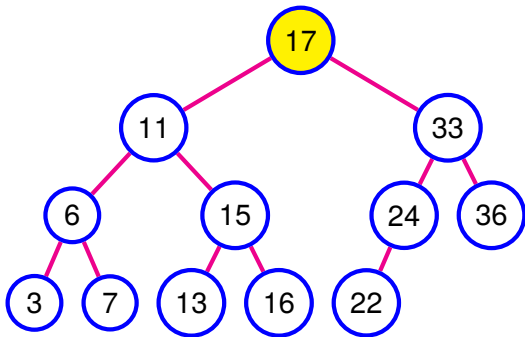
Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up



Tree walk example

- Start at the root of the tree
- Go down to the left child, then its left child, etc.
- When you can't go any deeper, go back up
- Then go down to the right child (then its *left* child, etc.)
- When you've done both sides, keep going back up
- After doing both sides of the root, you've visited every node



Tree walk pseudocode

```
Procedure DepthFirst(node) is
  if node is not None then
    visit(node);
    DepthFirst(node.left);
    DepthFirst(node.right)
  end
end
```

A note on pseudocode

- A *language-agnostic* tool for describing algorithms
- Focuses on the high-level logic, can omit low-level details
- Once you get the logic right, *translate* to your prog. lang.
 - ▶ You'll also have to fill in the details that were left out!
 - ▶ Not always trivial

Tree walk pseudocode

```
Procedure DepthFirst(node) is
  if node is not None then
    visit(node);
    DepthFirst(node.left);
    DepthFirst(node.right)
  end
end
```

- Visit, then recursively traverse the left subtree, then recursively traverse the right subtree
 - ▶ *left subtree* = the left child of a node, plus its descendants (m.m. right)
- This algorithm is called *depth-first search* (DFS)
 - ▶ We go as deep as we can, then we backtrack

Tree walkS (plural)

If all you want is to call `visit` on every node, we're done.

- E.g., summing up file sizes in a file system

But, some algorithms require visits to be in a specific order!

- Previous algorithm always visits parents before children
- But, e.g., interpreters require children to be visited before their parents! (Cf. Comp Sci 321)

Tree walkS (plural)

If all you want is to call `visit` on every node, we're done.

- E.g., summing up file sizes in a file system

But, some algorithms require visits to be in a specific order!

- Previous algorithm always visits parents before children
- But, e.g., interpreters require children to be visited before their parents! (Cf. Comp Sci 321)

For binary trees, $(2 + 1)!$ (= 6) possible orders of operations:

- | | | | | | |
|---------|---------|---------|---------|---------|---------|
| • visit | • left | • left | • visit | • right | • right |
| • left | • visit | • right | • right | • visit | • left |
| • right | • right | • visit | • left | • left | • visit |

Reordering the body gives you depth-first search variants.

Tree walkS (plural)

#1	#2	#3	#4	#5	#6
• visit	• left	• left	• visit	• right	• right
• left	• visit	• right	• right	• visit	• left
• right	• right	• visit	• left	• left	• visit

- #1 (original) is called *pre-order* depth-first search: visits parents *before* (pre) children
- #2 is called *in-order*: visits parents *in-between* children
- #3 is called *post-order*: visits parents *after* (post) children
 - ▶ This is the one we want for interpreters
- #4–6 are the same, but right-to-left: *reverse* pre-order, etc.
 - ▶ Reverse post-order used a lot in program analysis (compilers, security)

Tree walkS (plural)

#1	#2	#3	#4	#5	#6
• visit	• left	• left	• visit	• right	• right
• left	• visit	• right	• right	• visit	• left
• right	• right	• visit	• left	• left	• visit

- #1 (original) is called *pre-order* depth-first search: visits parents *before* (pre) children
- #2 is called *in-order*: visits parents *in-between* children
- #3 is called *post-order*: visits parents *after* (post) children
 - ▶ This is the one we want for interpreters
- #4–6 are the same, but right-to-left: *reverse* pre-order, etc.
 - ▶ Reverse post-order used a lot in program analysis (compilers, security)

For general k -ary trees, $(k + 1)!$ possible orders.

- But you'll most likely see (reverse) pre- and post-order

Tree walk on rose trees

```
Procedure RosePreOrder(node) is
|   visit(node);
|   for c in node.children do
|       |   RosePreOrder(c)
|   end
end
```

- Iterate over all children, traversing each one recursively
- Only (reverse) pre- and post- order really make sense here

Stop! Question time!

Before we move on to general graph search...

Anything unclear so far?

Anything I missed?

Graph search

Graph search

Similar idea to tree walks

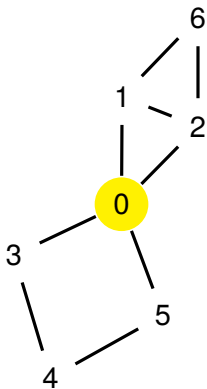
- Want to perform an action at every node of a graph.
- We'll use the same pattern to accomplish other tasks too!

But with general graphs, we now have to handle cycles!

- Will need to keep track of where we've been.
- So we don't go around in circles.

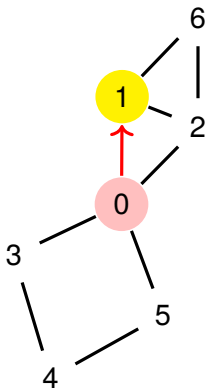
Let's start from our rose tree depth-first search and adapt it.

Graph DFS example



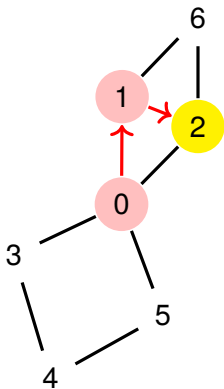
Graphs don't have roots; have to pick a starting point.

Graph DFS example

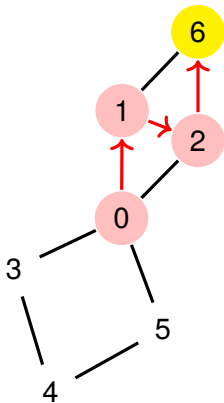


A node's neighbors/successors are not ordered.
Traversing in any order is ok.

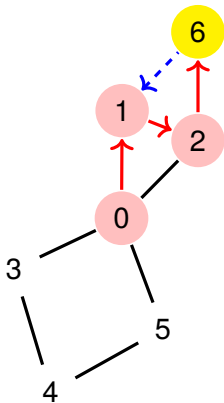
Graph DFS example



Graph DFS example

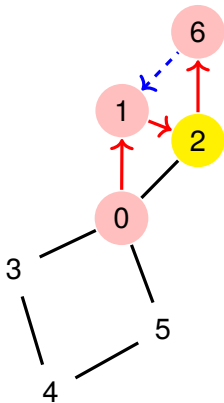


Graph DFS example

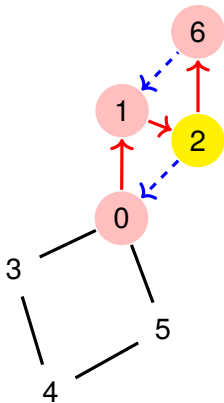


Been there before. No need to go there again. Backtrack.

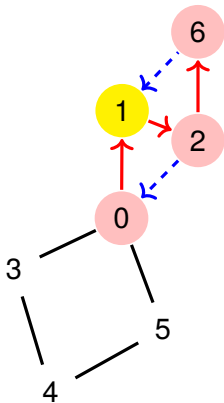
Graph DFS example



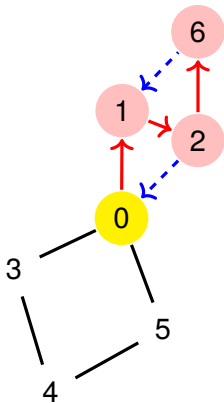
Graph DFS example



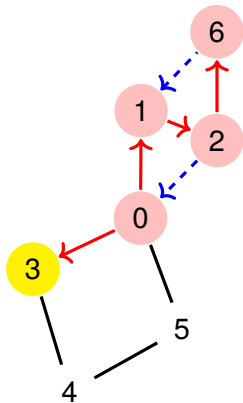
Graph DFS example



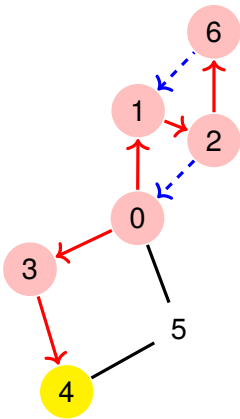
Graph DFS example



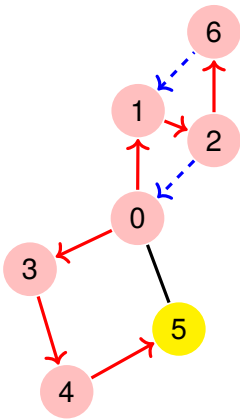
Graph DFS example



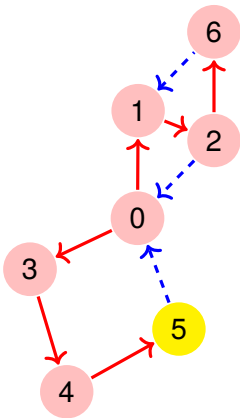
Graph DFS example



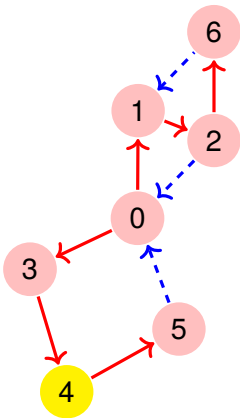
Graph DFS example



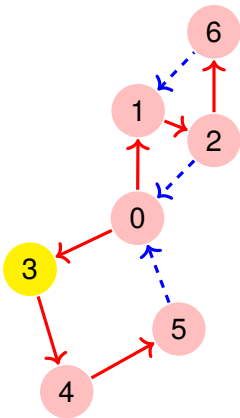
Graph DFS example



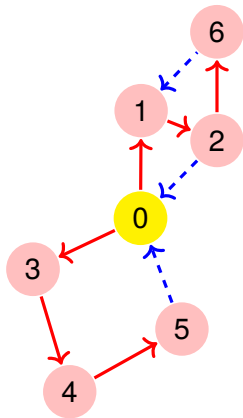
Graph DFS example



Graph DFS example

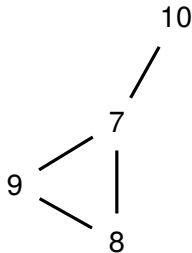
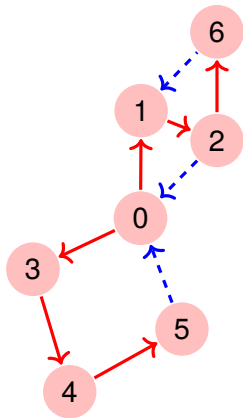


Graph DFS example



Once we've explored all the neighbors of the start, we're done.

Graph DFS example



Except we've only visited the nodes *reachable* from the start!
That's generally what we want with graph search.

Recursive DFS algorithm

Procedure DFS(*graph*, *start*) is

seen $\leftarrow$ new array of size $|V|$, filled with false;

 Procedure Traverse(*v*) is

 if not *seen*[*v*] then

seen[*v*] $\leftarrow$ true;

 visit(*v*);

 for *u* in *graph*.get_succs(*v*) do

 Traverse(*u*)

 end

 end

 end

 Traverse(*start*);

 return *seen*

end

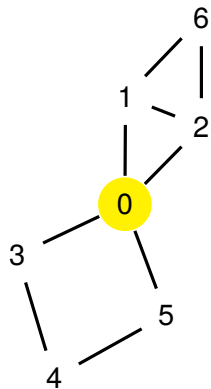
Similar to rose tree version, but recording where you've been.

And whoever calls DFS picks the starting point.

Recursive DFS algorithm (records paths)

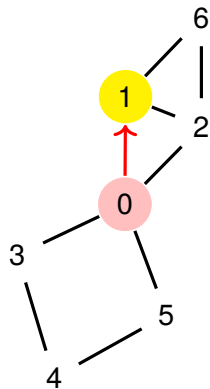
- If all you want to do is visit nodes, this is all you need.
 - ▶ But graph search can do much more than that!
- What if you're looking for paths between nodes?
 - ▶ We know whether we've reached a node or not, but we don't keep track of how we got there!
- Thankfully, doing that is not any harder.
 - ▶ Instead of using an array of booleans (seen)
 - ▶ Use an array of node ids (preds)
 - ▶ And record where we *came from* when we get to a node

Recursive DFS algorithm (records paths)



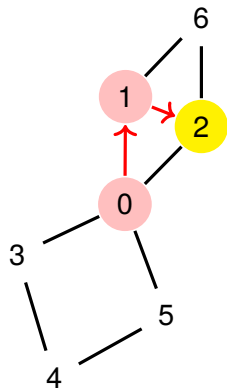
0	1	2	3	4	5	6
0						

Recursive DFS algorithm (records paths)



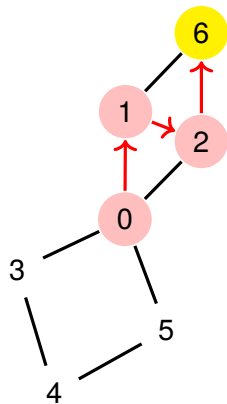
0	1	2	3	4	5	6
0	0					

Recursive DFS algorithm (records paths)



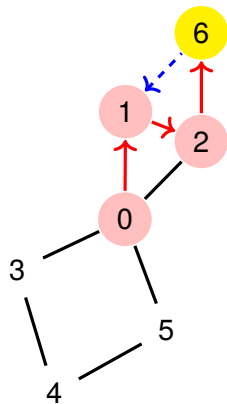
0	1	2	3	4	5	6
0	0	1				

Recursive DFS algorithm (records paths)



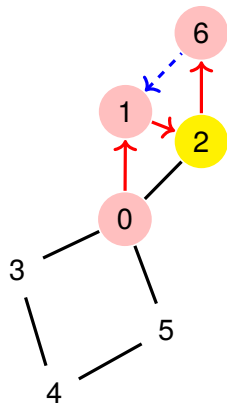
0	1	2	3	4	5	6
0	0	1				2

Recursive DFS algorithm (records paths)



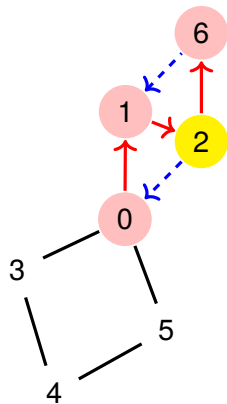
0	1	2	3	4	5	6
0	0	1				2

Recursive DFS algorithm (records paths)



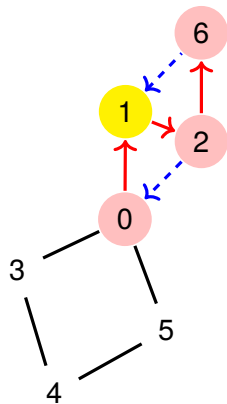
0	1	2	3	4	5	6
0	0	1				2

Recursive DFS algorithm (records paths)



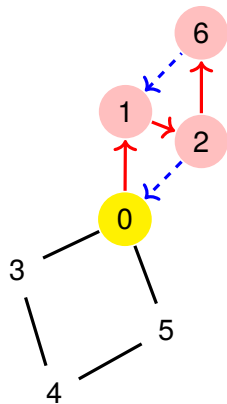
0	1	2	3	4	5	6
0	0	1				2

Recursive DFS algorithm (records paths)



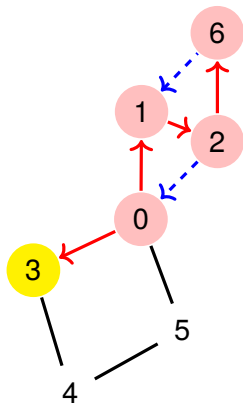
0	1	2	3	4	5	6
0	0	1				2

Recursive DFS algorithm (records paths)



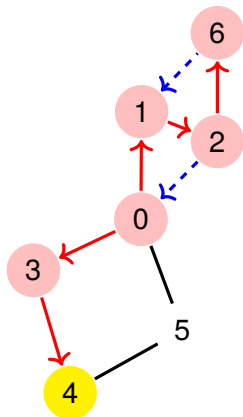
0	1	2	3	4	5	6
0	0	1				2

Recursive DFS algorithm (records paths)



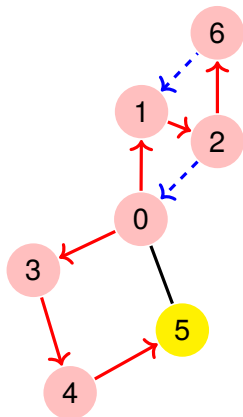
0	1	2	3	4	5	6
0	0	1	0			2

Recursive DFS algorithm (records paths)



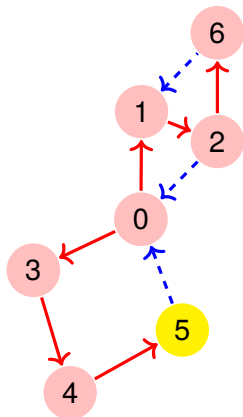
0	1	2	3	4	5	6
0	0	1	0	3		2

Recursive DFS algorithm (records paths)



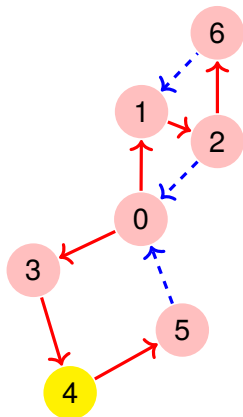
0	1	2	3	4	5	6
0	0	1	0	3	4	2

Recursive DFS algorithm (records paths)



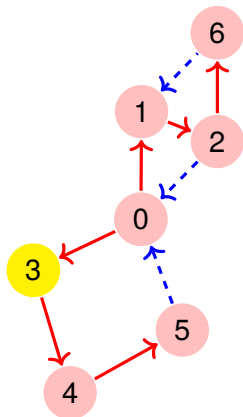
0	1	2	3	4	5	6
0	0	1	0	3	4	2

Recursive DFS algorithm (records paths)



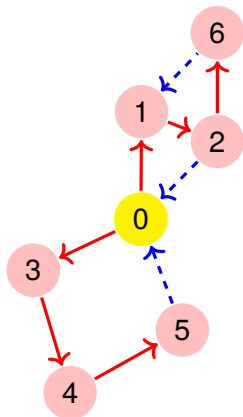
0	1	2	3	4	5	6
0	0	1	0	3	4	2

Recursive DFS algorithm (records paths)



0	1	2	3	4	5	6
0	0	1	0	3	4	2

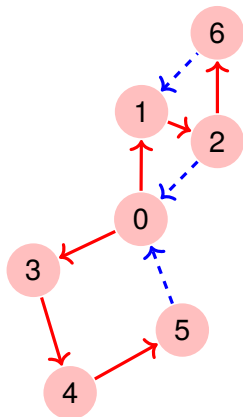
Recursive DFS algorithm (records paths)



Reverse tree, children point to parents.
Each node has just the one parent.
Can store in a vector!

0	1	2	3	4	5	6
0	0	1	0	3	4	2

Recursive DFS algorithm (records paths)



0	1	2	3	4	5	6
0	0	1	0	3	4	2

Reverse tree, children point to parents.
Each node has just the one parent.

Can store in a vector!

How do we go from 0 to 6?
Work backwards from 6 in the array!
Gives **a** path.

Not necessarily the most direct!

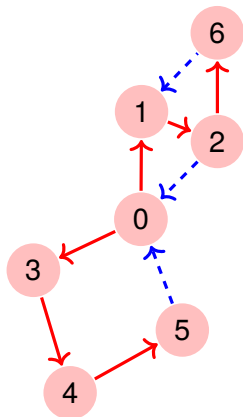
QUIZ: what **is** that path?

A: 0-1-6

B: 0-2-6

C: 0-1-2-6

Recursive DFS algorithm (records paths)



0	1	2	3	4	5	6
0	0	1	0	3	4	2

Reverse tree, children point to parents.
Each node has just the one parent.

Can store in a vector!

How do we go from 0 to 6?
Work backwards from 6 in the array!
Gives **a** path.

Not necessarily the most direct!

QUIZ: what **is** that path?

A: 0-1-6

B: 0-2-6

C: 0-1-2-6

Recursive DFS algorithm (records paths)

```
Procedure FindPath(graph, start) is
  | preds  $\leftarrow$  new array of size  $|V|$ , filled with false;
  |
  | Procedure Traverse(pred, v) is
  | | if not preds[v] then
  | | | preds[v]  $\leftarrow$  pred;
  | | | for u in graph.get_succs(v) do
  | | | | Traverse(v, u)
  | | | end
  | | end
  | end
  | Traverse(start, start);
  | return preds
end
```

2 arguments (*pred* and *v*) updated in lock-step.

Analogous to linked list traversal (*prev* and *curr*).

DFS Time Complexity

Visit each vertex at most once: $\mathcal{O}(|V|)$ (assuming $\mathcal{O}(1)$ visit)

For each vertex, loop over successors.

Total number of successors, across all vertices: $\mathcal{O}(|E|)$

DFS Time Complexity

Visit each vertex at most once: $\mathcal{O}(|V|)$ (assuming $\mathcal{O}(1)$ visit)

For each vertex, loop over successors.

Total number of successors, across all vertices: $\mathcal{O}(|E|)$

Adjacency list

- Iterating over successors, all vertices: $\mathcal{O}(|E|)$
- **Total:** $\mathcal{O}(|V|) \times \mathcal{O}(1) + \mathcal{O}(|E|) = \mathcal{O}(|V| + |E|)$

DFS Time Complexity

Visit each vertex at most once: $\mathcal{O}(|V|)$ (assuming $\mathcal{O}(1)$ visit)

For each vertex, loop over successors.

Total number of successors, across all vertices: $\mathcal{O}(|E|)$

Adjacency list

- Iterating over successors, all vertices: $\mathcal{O}(|E|)$
- **Total:** $\mathcal{O}(|V|) \times \mathcal{O}(1) + \mathcal{O}(|E|) = \mathcal{O}(|V| + |E|)$

Adjacency matrix

- Iterating over successors, one vertex: $\mathcal{O}(|V|)$
- **Total:** $\mathcal{O}(|V|) \times \mathcal{O}(1) + \mathcal{O}(|V|) \times \mathcal{O}(|V|) = \mathcal{O}(|V|^2)$

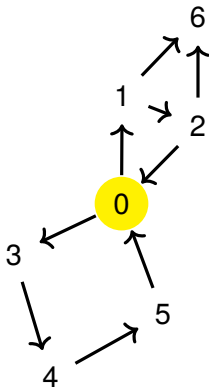
Stop! Question time!

Before we move on to one more application of graph search...

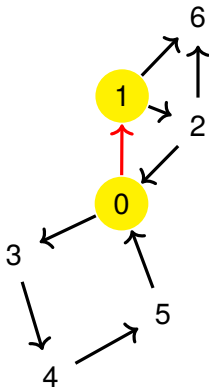
Anything unclear so far?

Anything I missed?

DFS for cycle detection

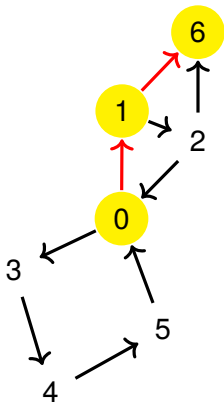


DFS for cycle detection



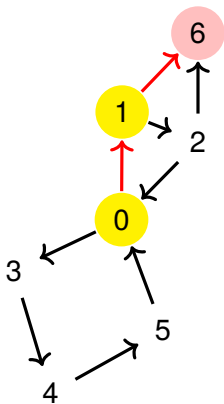
Keep track of the nodes we've started exploring.

DFS for cycle detection



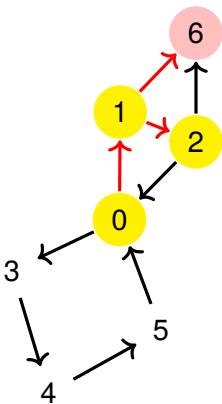
Keep track of the nodes we've started exploring.

DFS for cycle detection



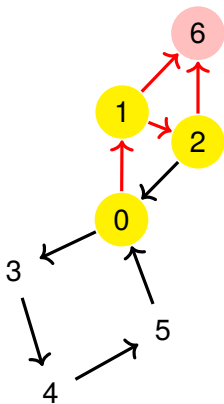
Keep track of the nodes we've started exploring.
When we're done with their neighbors, mark as finished.

DFS for cycle detection



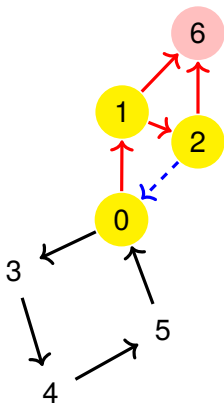
Keep track of the nodes we've started exploring.
When we're done with their neighbors, mark as finished.

DFS for cycle detection



Keep track of the nodes we've started exploring.
When we're done with their neighbors, mark as finished.
Seeing finished nodes again is ok.

DFS for cycle detection



Keep track of the nodes we've started exploring.
When we're done with their neighbors, mark as finished.
Seeing finished nodes again is ok.
Seeing an unfinished node again means we have a cycle!

DFS for cycle detection

Procedure FindCycle(*graph*) is

started $\leftarrow$ new array of size $|V|$, filled with false;

finished $\leftarrow$ new array of size $|V|$, filled with false;

 Procedure Traverse(*v*) is

 if not *finished*[*v*] then

 if *started*[*v*] then

 we found a cycle!

 end

started[*v*] $\leftarrow$ true;

 for *u* in *graph*.get_succs(*v*) do

 Traverse(*u*)

 end

finished[*v*] $\leftarrow$ true;

 end

 end

 for *v* in *graph*.get_vertices() do

 Traverse(*v*)

 end

end

Breadth-first search (BFS)

All our traversals so far have been *depth-first*

- I.e., go as far along one path as possible, then backtrack
- Then go as far as possible along another path, etc.

Breadth-first search (BFS)

All our traversals so far have been *depth-first*

- I.e., go as far along one path as possible, then backtrack
- Then go as far as possible along another path, etc.

Not all visiting orders can be achieved in a depth-first manner!
Here's an interesting one that can't.

- Start from one vertex
- Visit all its neighbors
- *Then* visit all *their* neighbors
- Continue with nodes 3 edges away from start, then 4, etc.
- This is *breadth-first* order: instead of going deep, go wide

That order is necessary for some algorithms!

- Two-space garbage collection (Cf. Comp Sci 321)
- “Forest fire” algorithm (graphics, Cf. Comp Sci 351)

Breadth-first search (BFS)

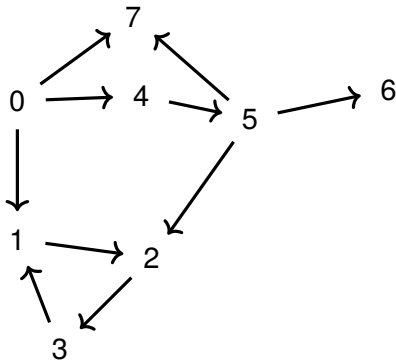
To get that order, we'll need a new traversal template.

Insight: depth-first search is *recursive*

- I.e., it uses a *stack* to keep track of what to do next
 - ▶ What stack? The call stack! Implicit in recursive calls
- Last-in, first-out. When we reach a dead end, we go back to the last node we saw (parent)

To get breadth-first order, what can we use instead?

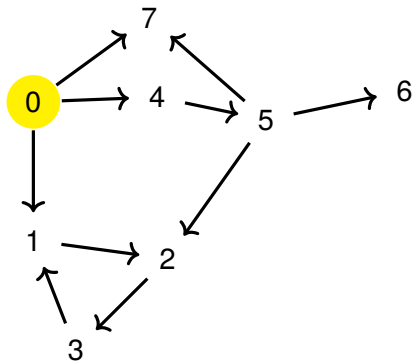
Running BFS on a digraph



Queue: 0

0	1	2	3	4	5	6	7

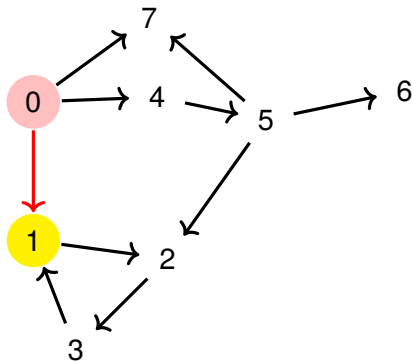
Running BFS on a digraph



Queue: $\langle 1, 4, 7 \rangle$

0	1	2	3	4	5	6	7
0							

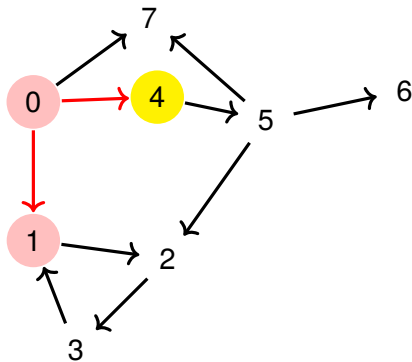
Running BFS on a digraph



Queue: $\langle 4, 7, 2 \langle$

0	1	2	3	4	5	6	7
0	0						

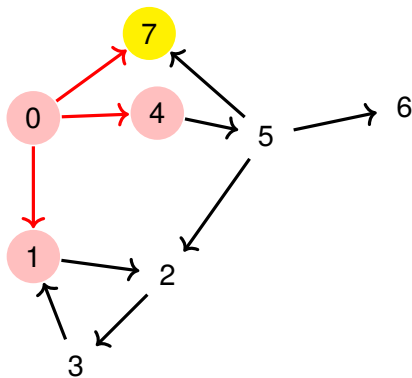
Running BFS on a digraph



Queue: $\langle 7, 2, 5 \langle$

0	1	2	3	4	5	6	7
0	0			0			

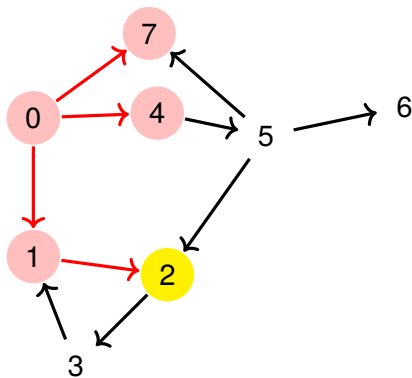
Running BFS on a digraph



Queue: $\langle 2, 5 \rangle$

0	1	2	3	4	5	6	7
0	0			0			0

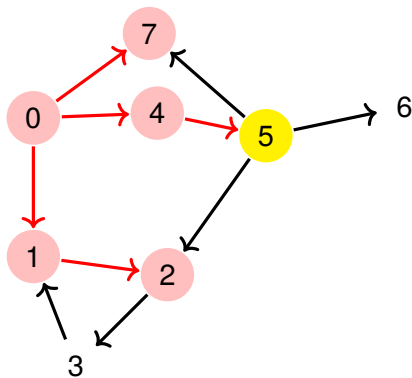
Running BFS on a digraph



Queue: $\langle 5, 3 \langle$

0	1	2	3	4	5	6	7
0	0	1		0			0

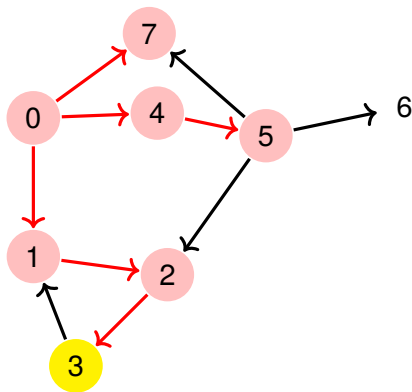
Running BFS on a digraph



Queue: $\langle 3, 6 \rangle$

0	1	2	3	4	5	6	7
0	0	1		0	4		0

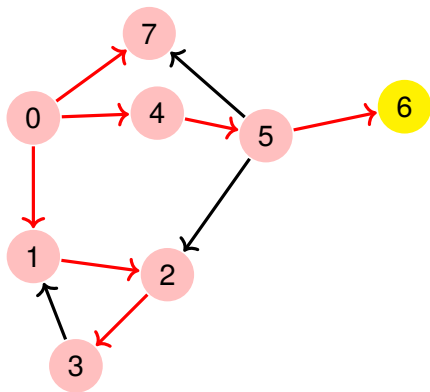
Running BFS on a digraph



Queue: 6

0	1	2	3	4	5	6	7
0	0	1	2	0	4		0

Running BFS on a digraph

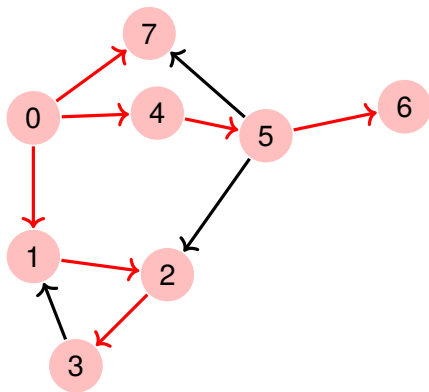


Queue:

<<

0	1	2	3	4	5	6	7
0	0	1	2	0	4	5	0

Running BFS on a digraph



Queue:

<<

0	1	2	3	4	5	6	7
0	0	1	2	0	4	5	0

BFS algorithm

Procedure **BFS**(*graph*, *start*) is

```
  seen  $\leftarrow$  new array of size  $|V|$ , filled with false;  
  todo  $\leftarrow$  new queue;  
  todo.enqueue(start);  
  while todo is not empty do  
    v  $\leftarrow$  todo.dequeue();  
    if not seen[v] then  
      visit(v);  
      seen[v]  $\leftarrow$  true;  
      for u in graph.get_succs(v) do  
        todo.enqueue(u)  
      end  
    end  
  end  
end
```

Same core as DFS.

But using a queue instead of recursion.

Fun fact about BFS

If your graph is unweighted, the paths you get by reading *preds* backwards are the shortest paths from the start to each node!

If you instead have a weighted graph, “shortest” means “smallest sum of weights”.

For that, you’ll need a more advanced algorithm (stay tuned).

BFS Time Complexity

Same thing as DFS

- Adjacency list: $\mathcal{O}(|V| + |E|)$
- Adjacency matrix: $\mathcal{O}(|V|^2)$

Assuming $\mathcal{O}(1)$ queue operations (we've seen how to get that).

In general, algorithms that follow the pattern “do something for every vertex and every edge” will have that complexity.

BFS Space Complexity

BFS uses a queue: *auxiliary storage*

I.e., data it needs beyond its input

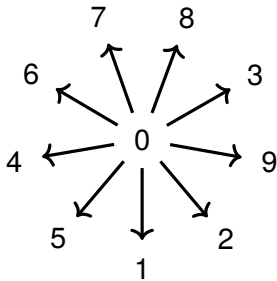
How big can that queue get?

BFS Space Complexity

BFS uses a queue: *auxiliary storage*

I.e., data it needs beyond its input

How big can that queue get?



In the worst case, $\mathcal{O}(|V|)$

Same for DFS's stack (but worst case graph is a straight line)

DFS vs BFS

Let's rewrite DFS to use an explicit stack instead of recursion.

Procedure DFS(*graph*, *start*) is

```
    seen ← new array;  
    todo ← new stack;  
    todo.push(start);  
    while todo is not empty do  
        | v ← todo.pop();  
        | if not seen[v] then  
        | | visit(v);  
        | | seen[v] ← true;  
        | | for u in  
        | | | graph.get_succs(v)  
        | | | do  
        | | | | todo.push(u)  
        | | | end  
        | | end  
        | end  
    end  
end
```

Procedure BFS(*graph*, *start*) is

```
    seen ← new array;  
    todo ← new queue;  
    todo.enqueue(start);  
    while todo is not empty do  
        | v ← todo.dequeue();  
        | if not seen[v] then  
        | | visit(v);  
        | | seen[v] ← true;  
        | | for u in  
        | | | graph.get_succs(v)  
        | | | do  
        | | | | todo.enqueue(u)  
        | | | end  
        | | end  
        | end  
    end  
end
```

Generic graph search

Procedure `Search(graph, start)` is

```
seen ← new array;  
todo ← new to-do-list;  
todo.add(start);  
while todo is not empty do  
    v ← todo.remove();  
    if not seen[v] then  
        visit(v);  
        seen[v] ← true;  
        for u in graph.get_succs(v) do  
            todo.add(u)  
        end  
    end  
end  
end
```

If *todo* is a stack we get DFS.
If *todo* is a queue we get BFS.

If *todo* is a something else
we get a different search.

E.g., priority queue + heuristic → A^*
Priority queue → coming soon!
 A^* → take Comp Sci 348

More examples

- <https://www.cs.usfca.edu/~galles/visualization/DFS.html>
- <https://www.cs.usfca.edu/~galles/visualization/BFS.html>

Next time: Priority queues